
Documentación Técnica: Proyecto EdgeLeak AI

Fase 2: Seguridad, Gestión de Usuarios y Pruebas Unitarias

1. Evolución de la Arquitectura de Datos (Migración a v4)

Para soportar el sistema de administración y el motor de búsqueda, el esquema relacional local (Sqlite) fue refactorizado y migrado de la versión 3 a la versión 4 (edgeleak_v4.db).

- Tabla Usuarios: Se mantiene el campo lógico es_temporal (INTEGER 0/1) para el flujo de cambio obligatorio de contraseña.
- Gestión de Roles: Se integraron campos para diferenciar privilegios entre Administradores y Operadores, permitiendo un control de acceso granular en la interfaz.
- Integridad de Datos: El campo correo se definió como UNIQUE para evitar duplicidad de registros durante la fase de expansión del sistema.

2. Módulo CRUD Administrativo Completo

Se desarrolló la pantalla AdminUsersScreen, la cual centraliza la gestión de cuentas y privilegios del sistema:

- Listado Reactivo: Implementación de ListenableBuilder para sincronizar la UI con la lista de usuarios del AuthController de manera eficiente.
- Eliminación Segura: El sistema incluye una capa de validación que impide que un administrador elimine su propia cuenta activa. El método eliminarUsuario(id) verifica la sesión antes de ejecutar el DELETE.
- Identificación por Roles: Uso de avatares con códigos de color (Rojo para administradores, Azul para operadores) para facilitar la auditoría visual rápida.

3. Motor de Búsqueda y Optimización de UX

Se añadió una funcionalidad de búsqueda dinámica para garantizar la escalabilidad y facilidad de uso del aplicativo:

- Filtrado en Memoria: Para evitar latencia y consumo excesivo de batería, el filtrado se realiza directamente sobre los datos cargados mediante el predicado .where().
 - Hotfix de Renderizado: Se corrigió el error crítico Invalid constant value (dart: invalid_constant) eliminando los prefijos const en los widgets de texto que dependen de la variable dinámica _filter.
 - Normalización de Búsqueda: El algoritmo utiliza toLowerCase() en ambos lados de la comparación para asegurar resultados precisos independientemente del uso de mayúsculas.
-

4. Protocolo de Recuperación y Seguridad SMTP

El sistema integra una solución robusta de recuperación de acceso conectándose directamente a la infraestructura de Google Workspace:

- Cifrado SMTPS: Uso de SSL/TLS implícito para garantizar el cifrado de extremo a extremo durante la transmisión de credenciales.
 - Generación Criptográfica: El método `generarContrasenaSegura()` emplea la librería `dart:math` para inyectar obligatoriamente mayúsculas, minúsculas, números y símbolos en cadenas de 12 caracteres.
 - Interceptación de UX: Uso de `PopScope(canPop: false)` para forzar al usuario a actualizar contraseñas temporales antes de acceder al Dashboard.
-

5. Estrategia de Pruebas Unitarias (TDD)

Bajo la metodología de Desarrollo Guiado por Pruebas, se audita la lógica interna para garantizar la mantenibilidad:

- Pruebas de Lógica (`user_logic_test.dart`): Validación de la máquina de estados `RegExp` para fortaleza de contraseñas y flujos de autenticación.
 - Simulación de Caudal: Evaluación de cálculos matemáticos en el `DashboardController` para detectar estados de "Anomalía" y "Fuga".
 - Manejo de Memoria: Aplicación del método `addTearDown()` para eliminar instancias activas de `Timers` y evitar fugas de memoria (`Memory Leaks`) en la Dart VM.
-

6. Detalles de Implementación: Arquitectura y Lógica Interna

Esta sección detalla las decisiones técnicas tomadas para optimizar el rendimiento y la seguridad del módulo de administración:

- Patrón de Gestión de Estado: Se migró de un manejo de estado tradicional a un `ListenableBuilder` vinculado al `AuthController`, lo que garantiza que solo se reconstruyan los elementos necesarios de la UI al filtrar o eliminar usuarios.
 - Algoritmo de Filtrado Iterativo: La lógica de búsqueda procesa la colección de objetos `UsuarioModel` en memoria, proporcionando una experiencia de usuario fluida (60 FPS) durante la escritura sin peticiones asíncronas constantes a la DB.
 - Validación de Integridad en el Borrado: El proceso incluye un flujo de confirmación asíncrono que captura excepciones y las propaga mediante un `ScaffoldMessenger`, informando al administrador sobre cualquier restricción de seguridad.
 - Refactorización de Widgets Dinámicos: Se transformaron widgets estáticos en estructuras dinámicas para que mensajes de estado (como "No se encontraron coincidencias") se actualicen reactivamente según el contenido del controlador de búsqueda.
-